

Разработка драйверов в Linux, 1

Владислав Богданов

2026-06-16

Оглавление

1. Создание Makefile	1
2. Оформление кода модуля	2
3. Особенности отладки модуля (ИИ инфо)	3
3.1. printk + dmesg	3
3.2. /proc и /sys — "карманы" для отладочных данных	5
3.3. Анализ падений: oops и panic	5
3.4. ftrace — трассировка без пересборки	6
3.5. kprobes — "жучки" в любом месте ядра	6
3.6. Тяжелая артиллерия: kgdb + GDB (удаленная отладка)	7
3.7. Специализированные инструменты	7
4. Инструментарий	8
5. Управление модулем	8
6. Kernel symbols	9
7. Домашнее задание	9

Материал написан на основе курса "Разработка модулей ядра Linux (Linux Kernel developer)" от <https://inzhennerka.tech/> и собранных данных в интернете.

1. Создание Makefile

Для работы Makefile нужна честная табуляция, а не 4 пробела как делают многие редакторы.

Параметры:

CURRENT = **\$(shell uname -r)** - текущая версия ядра

KERNEL_DIR = **/lib/modules/&(CURRENT)/build** - путь до директории с библиотеками необходимыми для сборки, этот путь всегда одинаков в Linux

PWD = **\$(shell pwd)** - путь до текущей директории

obj-m := hello.o - указываем какие модули собираются, имя всегда с маленькой буквы и если имя состоит из нескольких слов, пробел меняем на нижнее подчеркивание. Сам модуль будет kernel object (.ko).

default: - сборка

clean - что бы удалить используем только clean, так как rm и т.п. не поможет, так как будет много скрытых файлов.

Пример Makefile

```
user@ubuntu:~/prj/linux$ cat Makefile
CURRENT = $(shell uname -r)
KERNEL_DIR = /lib/modules/$(CURRENT)/build
PWD = $(shell pwd)

obj-m := hello.o

default:
    $(MAKE) -C $(KERNEL_DIR) M=$(PWD) modules

clean:
    $(MAKE) -C $(KERNEL_DIR) M=$(PWD) clean
```

Пример Makefile из нескольких файлов

```
CURRENT = $(shell uname -r)
KERNEL_DIR = /lib/modules/$(CURRENT)/build
PWD = $(shell pwd)

obj-m := hello.o
```

```
hello-objs := start.o stop.o my_alert.o

default:
    $(MAKE) -C $(KERNEL_DIR) M=$(PWD) modules

clean:
    $(MAKE) -C $(KERNEL_DIR) M=$(PWD) clean
```

2. Оформление кода модуля

Код можно писать по разному.

Linux Kernel Coding Style - свод правил написания кода

Примеры:

- не приняты строчные комментарии
- goto можно использовать
- и т.д.

Библиотеки :

- `<linux/module.h>`
- есть своя библиотека для работы со строками

Функции создаем:

- в качестве параметров указываем (void) в функциях без аргументов
- Статичные, что бы не пересекались с другими.
- Названия произвольные
- Возвращают intc системным подходом (0 - ОК, -value - ошибки), тип значения определен errno (error.h)
- Выгружающая функция типа void, рекомендовалось ранее писать return;

Классический вариант кода

```
#include <linux/module.h>

/*
обязательно указываем тип лицензии
*/

MODULE_LICENSE("GPL");
/*
необязательный макрос
*/
```

```

MODULE_AUTHOR("user");

/*
*/
static int __init hello_init(void)
{
    printk(KERN_ALERT "module started\n");
    return 0;
}

/*
*/
static void __exit hello_exit(void)
{
    printk(KERN_ALERT "module exited\n");
    return;
}

module_init(hello_init);
module_exit(hello_exit);

```

3. Особенности отладки модуля (ИИ инфо)

Отладка модулей ядра — это совсем другая философия. Здесь нет gdb по умолчанию, нет printf в консоль, и одна ошибка может положить всю систему с потерей несохраненных данных. Отладку логики удобнее отладить отдельно, в потом перенести в код модуля ядра.

3.1. printk + dmesg

Использование printk

```

printk("<1> module started\n");
что идентично
printk(KERN_ALERT "module started\n")
`\n` - обязателен, что бы сразу произошел вывод

или

#include <linux/module.h>
#include <linux/kernel.h>
#include <linux/init.h>

static int __init my_module_init(void)
{
    // pr_info, pr_err, pr_debug – современные обёртки над printk
    pr_info("my_module: загружается\n");

```

```

    pr_debug("my_module: отладочное сообщение\n"); // появится только если включён
DEBUG
    pr_err("my_module: ошибка!\n");
    return 0;
}

static void __exit my_module_exit(void)
{
    pr_info("my_module: выгружается\n");
}

module_init(my_module_init);
module_exit(my_module_exit);
MODULE_LICENSE("GPL");

```

Просмотр логов:

```

# Все сообщения ядра
dmesg

# Только ошибки
dmesg --level=err

# Следить в реальном времени (очень удобно!)
dmesg -w

# Очистить буфер перед тестом
dmesg -c

```

Важные уровни логирования (от 0 до 7):

- KERN_EMERG (0) — система мертва
- KERN_ERR (3) — ошибки
- KERN_WARNING (4) — предупреждения
- KERN_INFO (6) — информация
- KERN_DEBUG (7) — отладка

```

Включи динамическую отладку, чтобы не пересобирать ядро:
# Включаем pr_debug для конкретного модуля на лету
_echo 'module my_module +p' > /sys/kernel/debug/dynamic_debug/_

```

3.2. /proc и /sys — "карманы" для отладочных данных

Когда `printk` уже не хватает (например, нужно посмотреть состояние структуры в реальном времени), создай свой файл в `/proc`:

```
#include <linux/proc_fs.h>
#include <linux/seq_file.h>

static int my_proc_show(struct seq_file *m, void *v)
{
    seq_printf(m, "counter = %d\n", my_counter);
    seq_printf(m, "state = %s\n", my_state ? "active" : "idle");
    return 0;
}

static int __init my_init(void)
{
    proc_create_single("my_module_debug", 0444, NULL, my_proc_show);
    return 0;
}
```

Теперь в user-space:

```
cat /proc/my_module_debug
```

3.3. Анализ падений: oops и panic

Когда модуль делает что-то не то (разыменование `NULL`, доступ к чужой памяти), ядро генерирует oops — отчет о катастрофе.

Что делать при oops:

```
# 1. Увеличиваем детализацию
echo 1 > /proc/sys/kernel/panic_on_oops # паниковать сразу (для отладки)
echo 7 > /proc/sys/kernel/printk        # максимум логов

# 2. Смотрим символы
cat /proc/kallsyms | grep my_function

# 3. Декодируем адрес в имя функции
# Если в oops видно "RIP: 0010:my_function+0x2a/0x50 [my_module]"
# используем утилиту из исходников ядра:
scripts/decode_stacktrace.sh vmlinux < oops.txt
```

Полезный инструмент — `kmemleak` (ищет утечки памяти в ядре):

```
# Включаем в конфиге ядра: CONFIG_DEBUG_KMEMLEAK=y
echo scan > /sys/kernel/debug/kmemleak
cat /sys/kernel/debug/kmemleak
```

3.4. ftrace — трассировка без пересборки

Мощнейший инструмент, встроенный в ядро. Позволяет трассировать вызовы функций, смотреть latency, анализировать работу планировщика.

```
# Монтируем debugfs (если не смонтирован)
mount -t debugfs nodev /sys/kernel/debug

# Трассируем конкретную функцию
echo function > /sys/kernel/debug/tracing/current_tracer
echo my_module_function > /sys/kernel/debug/tracing/set_ftrace_filter
echo 1 > /sys/kernel/debug/tracing/tracing_on
cat /sys/kernel/debug/tracing/trace
```

3.5. kprobes — "жучки" в любом месте ядра

Позволяют поставить breakpoint на любую функцию ядра (даже не в твоём модуле) без пересборки.

```
#include <linux/kprobes.h>

static struct kprobe kp = {
    .symbol_name = "do_sys_open", // функция ядра, за которой следим
};

static int handler_pre(struct kprobe *p, struct pt_regs *regs)
{
    pr_info("do_sys_open вызван! IP=%lx\n", regs->ip);
    return 0;
}

static int __init my_init(void)
{
    kp.pre_handler = handler_pre;
    register_kprobe(&kp);
    return 0;
}
```

Есть еще **kretprobes** — ловят момент возврата из функции.

3.6. Тяжелая артиллерия: kgdb + GDB (удаленная отладка)

Это настоящий gdb для ядра. Требуется двух машин (или QEMU):

- Target — машина с отлаживаемым ядром
- Host — машина с gdb

Настройка через последовательный порт (самый частый случай): . В конфиге ядра:

```
CONFIG_KGDB=y
CONFIG_KGDB_SERIAL_CONSOLE=y
```

1. В boot-параметрах: kgdboc=ttyS0,115200 kgdbwait
2. На host-машине

```
gdb vmlinux
(gdb) target remote /dev/ttyS0
(gdb) break my_module_init
(gdb) continue
```

Альтернатива для разработчиков — QEMU + GDB:

```
# Запускаем ядро в QEMU с отладочным портом
qemu-system-x86_64 -kernel bzImage -append "..." -s -S

# В другом терминале
gdb vmlinux
(gdb) target remote localhost:1234
```

3.7. Специализированные инструменты

Инструмент	Для чего
lockdep	Поиск deadlock'ов в спинлоках
KASAN	Обнаружение use-after-free, buffer overflow (как AddressSanitizer)
UBSAN	Неопределенное поведение (переполнения, сдвиги)
kmemleak	Утечки памяти
perf	Профилирование производительности

bpfttrace / eBPF	Современная трассировка без вмешательства в код
------------------	---

Включаются в конфиге ядра:

```
CONFIG_KASAN=y
CONFIG_UBSAN=y
CONFIG_LOCKDEP=y
CONFIG_DEBUG_KMEMLEAK=y
```

Золотые правила отладки ядра

1. Разрабатывай в виртуалке! Одна ошибка — и хост-система в kernel panic. QEMU/VirtualBox спасут твои данные.
2. Компилируй ядро с отладочными опциями в режиме разработки:

```
CONFIG_DEBUG_INFO=y      # для gdb
CONFIG_KALLSYMS=y        # имена символов в oops
CONFIG_PRINTK=y
```

1. Используй make localmodconfig — собери ядро только с нужными модулями, это ускорит сборку в разы.
2. Смотри dmesg -w в соседнем терминале — это базовая гигиена.
3. Никогда не отлаживай на production-машине. Только на тестовой.
4. Читай Documentation/dev-tools/ в исходниках ядра — там вся актуальная документация.

4. Инструментарий

1. nm
2. modinfo

5. Управление модулем

Загрузить модуль:

```
sudo insmod ./hello.ko
```

Выгрузить модуль:

```
sudo rmmod ./hello.ko
```

6. Kernel symbols

Kernel symbols - понимаются названия функций в ядре. Все названия функций которые есть в ядро можно посмотреть в файле *kallsyms* в каталоге *proc*. Там перечислены все доступные программисту модуля символы в ядре. То есть драйвер может экспортировать свой символ (функцию) и превратить его доступным для любого другого модуля. Именно так в ядре модули взаимодействуют с друг другом. То есть можно использовать системные вызовы, но большого смысла обычно в этом нет. Как правило модули обращаются к друг другу через эти самые символы. Но раз есть возможность работать с функциями, то совершенно очевидно, что можно передавать в модули и работать с переменными, то есть можно создать то что называется **параметром модуля**.

Параметр модуля - переменная, которую можно передать в модуль при загрузке модуля и изменить в процессе работы модуля. Используются структуры категории *operations*.

```
cat /proc/kallsyms
```

7. Домашнее задание

1. Вывести из модуля ядра случайное число в некоем диапазоне при загрузке
2. Вывести из модуля ядра отформатированное текущее значение системного времени
3. Обратится из модуля к пространству пользователя, через специальную функцию, редкая и не рекомендуемая.